

MDSPAN

Document #: P0009r11
Date: 2021-05-11
Project: Programming Language C++
LWG
Reply-to: Christian Trott
<crtrott@sandia.gov>
D.S. Hollman
<me@dsh.fyi>
Damien Lebrun-Grandie
<lebrungrandt@ornl.gov>
Mark Hoemmen
<mhoemmen@stellarscience.com>
Daniel Sunderland
<dansunderland@gmail.com>
H. Carter Edwards
<hedwards@nvidia.com>
Bryce Adelstein Leibach
<bleibach@nvidia.com>
Mauro Bianco
<mbianco@cscs.ch>
Ben Sander
<ben.sander@amd.com>
Athanasios Iliopoulos
<athanasios.iliopoulos@nrl.navy.mil>
John Michopoulos
<john.michopoulos@nrl.navy.mil>
Nevin Liber
<nliber@anl.gov>

Contents

- 1 Revision History
 - 1.1 P0009r11: 2021-05 Mailing
 - 1.2 P0009r10: Pre 2020-02-Prague Mailing
 - 1.3 P0009r9: Pre 2019-02-Kona Mailing
 - 1.4 P0009r8: Pre 2018-11-SanDiego Mailing
 - 1.5 P0009r7: Post 2018-06-Rapperswil Mailing
 - 1.6 P0009r6 : Pre 2018-06-Rapperswil Mailing
 - 1.7 P0009r5 : Pre 2018-03-Jacksonville Mailing
 - 1.8 P0009r4 : Pre 2017-11-Albuquerque Mailing
 - 1.9 P0009r3 : Post 2016-06-Oulu Mailing
 - 1.10 P0009r2 : Pre 2016-06-Oulu Mailing
 - 1.11 P0009r1 : Pre 2016-02-Jacksonville Mailing
 - 1.12 P0009r0 : Pre 2015-10-Kona Mailing
 - 1.13 Related Activity
- 2 Description
- 3 Editing Notes
- 4 Wording
- 5 Next Steps
- 6 Implementation
- 7 Related Work

§ 1 Revision History

§ 1.1 P0009r11: 2021-05 Mailing

- Ask LEWG to poll on targeting P0009 for C++23
- Change all the sizes from `ptrdiff_t` to `size_t` and `index_type` to `size_type`, for consistency with `span` and the rest of the standard library`
- Renamed `IndexType` to `SizeType` or `SizeTypes` (depending if it is a single type or a parameter pack)
- Changed comparisons to hidden friends
- Explicitly mention which types are trivially copyable or empty. This is important as a major intended use case for this is heterogeneous computing. A trivially copyable is heavily used as a proxy for types which can be copied between a host (such as a CPU) and a device (such as a GPU) or between two devices by just copying the bytes which make up the object representation of the type. If they are not trivially copyable, heterogeneous computing would not be able to use these types as vocabulary types.
- State the conditions that make `basic_mdspan` trivially default constructible
- In `layout_*` types, made `operator()` and `stride()` constexpr
- In `layout_stride`, made assignment operators and `required_span_size()` constexpr to match the other `layout_*` types
- Renamed `subspan` to `submdspan`, as this only applies to `mdspan`
- Made `submdspan()` constexpr
- Tweak the wording of `is_strided`
- Renamed `all_type` to `full_extent_t` and `all` to `full_extent`
- Renamed `accessor_basic` to `default_accessor`
- Removed `accessor` policy `decay(p)` member function as it was an artifact from an earlier version of this proposal when `basic_mdspan` had a `span()` member function that returned a `std::span`
- Removed `span()` from `[mdspan.basic.members]` description as `.span()` was removed from an earlier version of this proposal

§ 1.2 P0009r10: Pre 2020-02-Prague Mailing

- Switched to mpark/wg21 pandoc format
- Add general description of `span` and `mdspan`
- Removed `mdspan_subspan` expo only type; use `basic_mdspan`<see below> instead
- Fixed typos in `accessor` table
- Made editorial changes to wording based on San Diego feedback
- Updated operational semantics subsection heading based on new style guidelines

§ 1.3 P0009r9: Pre 2019-02-Kona Mailing

- Wording fixes based on guidance: [LWG small group at 2018-11-SanDiego](#)

§ 1.4 P0009r8: Pre 2018-11-SanDiego Mailing

- Refinement based upon updated [prototype](#) / reference implementation

§ 1.5 P0009r7: Post 2018-06-Rapperswil Mailing

- wording reworked based on guidance: [LWG review at 2018-06-Rapperswil](#)
- usage of `span` requires reference to C++20 working draft
- namespace for library TS `std::experimental::fundamentals_v3`

§ 1.6 P0009r6 : Pre 2018-06-Rapperswil Mailing

P0009r5 was not taken up at 2018-03-Jacksonville meeting. Related [LEWG review of P0900 at 2018-03-Jacksonville meeting](#)

LEWG Poll We want the ability to customize the access to elements of `span` (ability to restrict, etc):

```
span<T, N, Accessor=...>
```

SF	F	N	A	SA
1	1	1	2	8

LEWG Poll We want the customization of `basic_mdspan` to be two concepts `Mapper` and `Accessor` (akin to `Allocator` design).

```
basic_mdspan<T, Extents, Mapper, Accessor>
mdspan<T, N...>
```

SF	F	N	A	SA
3	4	5	1	0

LEWG Poll: We want the customization of `basic_mdspan` to be an arbitrary (and potentially user-extensible) list of properties.

```
basic_mdspan<T, Extents, Properties...>
```

SF	F	N	A	SA
1	2	2	6	2

Changes from P0009r5 due to related LEWG reviews:

- Replaced variadic property list with *extents*, *layout mapping*, and *accessor* properties.
- Incorporated [P0454r1](#).
 - Added accessor policy concept.
 - Renamed `mdspan` to `basic_mdspan`.
 - Added a `mdspan` alias to `basic_mdspan`.

§ 1.7 P0009r5 : Pre 2018-03-Jacksonville Mailing

[LEWG review of P0009r4 at 2017-11-Albuquerque meeting](#)

LEWG Poll: We should be able to index with `span<int type[N]>` (in addition to array).

SF	F	N	A	SA
2	11	1	1	0

Against comment - there is not a proven needs for this feature.

LEWG Poll: We should be able to index with `1d mdspan`.

SF	F	N	A	SA
0	8	7	0	0

LEWG Poll: We should put the requirement on “`rank() <= N`” back to “`rank()==N`.”

Unanimous consent

LEWG Poll: With the editorial changes from small group, plus the above polls, forward this to LWG for Fundamentals v3.

Unanimous consent

Changes from P0009r4:

- Removed `nullptr` constructor.
- Added `constexpr` to indexing operator.
- Indexing operator requires that `rank()==sizeof...(indices)`.
- Fixed typos in examples and moved them to appendix.
- Converted note on how extensions to access properties may cause reference to be a proxy type to an “see below” to make it normative.

§ 1.8 P0009r4 : Pre 2017-11-Albuquerque Mailing

[LEWG review at 2017-03-Kona meeting](#)

[LEWG review of P0546r1 at 2017-03-Kona meeting](#)

LEWG Poll: Should we have a single template that covers both single and multi-dimensional spans?

SF	F	N	A	SA
1	6	2	6	3

Changes from P0009r3:

- Align with P0122r5 span [proposal](#).
- Rename to `mdspan`, multidimensional span, to align with `span`.
- Move preferred array extents mechanism to appendix.
- Expose codomain as a `span`.
- Add layout mapping concept.

§ 1.9 P0009r3 : Post 2016-06-Oulu Mailing

[LEWG review at 2016-06-Oulu](#)

LEWG did not like the name `array_ref`, and suggested the following alternatives: - `sci_span` - `numeric_span` - `multidimensional_span` - `multidim_span` - `mdspan` - `md_span` - `vla_span` - `multispan` - `multi_span`

LEWG Poll: Are member `begin()/end()` still good?

SF	F	N	A	SA
0	2	4	3	1

LEWG Poll: Want this proposal to provide range-producing functions outside `array_ref`?

SF	F	N	A	SA
0	1	3	2	3

LEWG Poll: Want a separate proposal to explore iteration design space?

SF	F	N	A	SA
9	1	0	0	0

Changes from P0009r2:

- Removed iterator support; a future paper will be written on the subject.
- Noted difference between multidimensional array versus language's array-of-array-of-array...
- Clearly describe requirements for the embedded type aliases (`element_type`, `reference`, etc).
- Expanded description of how the variadic properties list would work.
- Stopped allowing `array_ref<T[N]>` in addition to `array_ref<extents<N>>`.
- Clarified domain, codomain, and domain -> codomain mapping specifications.
- Consistently use *extent* and *extents* for the multidimensional index space.

§ 1.10 P0009r2 : Pre 2016-06-Oulu Mailing

[LEWG review at 2016-02-Jacksonville](#).

Changes from P0009r1:

- Adding details for extensibility of layout mapping.
- Move motivation, examples, and relaxed incomplete array type proposal to separate papers.
 - [P0331: Motivation and Examples for Polymorphic Multidimensional Array](#).
 - [P0332: Relaxed Incomplete Multidimensional Array Type Declaration](#).

§ 1.11 P0009r1 : Pre 2016-02-Jacksonville Mailing

[LEWG review at 2015-10-Kona](#).

LEWG Poll: What should this feature be called?

Name	#
view	5

span	9
array_ref	6
slice	6
array_view	6
ref	0
array_span	7
basic_span	1
object_span	3
field	0

LEWG Poll: Do we want 0-length static extents?

SF	F	N	A	SA
3	4	2	3	0

LEWG POLL: Do we want the language to support syntaxes like `x[3][][5]`?

Syntax	#
<code>view<int[3][0][5], property1></code>	12
<code>view<int, dimension<3, 0, dynamic_extent, 5>, property1></code>	4
<code>view<int[3][0][dynamic_extent][5], property1></code>	5
<code>view<int, 3, 0, dynamic_extent, 5, property1></code>	4
<code>view<int, 3, 0, dynamic_extent, 5, properties<property1>></code>	2
<code>view<arr<int, 3, 0, dynamic_extent, 5>, property1></code>	4
<code>view<int[3][0][5], properties<property1>></code>	9

LEWG POLL: Do we want the variadic property list in template args (either raw or in `properties<>`)? Note there is no precedence for this in the library.

SF	F	N	A	SA
3	6	3	0	0

LEWG POLL: Do we want the per-view bounds-checking knob?

SF	F	N	A	SA
3	4	1	2	1

Changes from P0009r0:

- Renamed `view` to `array_ref`.
- How are users allowed to add properties? Needs elaboration in paper.
- `view<int[][]>::layout` should be named.
- Rename `is_regular` (possibly to `is_affine`) to avoid overloading the term with the Regular concept.
- Make static `span()`, `operator()`, `constructor`, etc variadic.
- Demonstrate the need for improper access in the paper.
- In `operator()`, take integral types by value.

§ 1.12 P0009r0 : Pre 2015-10-Kona Mailing

Original non-owning multidimensional array reference (`view`) paper with motivation, specification, and examples.

§ 1.13 Related Activity

Related [LEWG review of P0546r1 at 2017-11-Albuquerque meeting](#)

LEWG Poll: `span` should specify the dynamic extent as the element type of the first template parameter rather than the (current) second template parameter

SF	F	N	A	SA
5	3	2	2	0

LEWG Poll: `span` should support the addition of access properties variadic template parameters

SF	F	N	A	SA
----	---	---	---	----

Authors agreed to bring a separate paper ([P0900r0]) discussing how the variadic properties will work.

§ 2 Description

The proposed polymorphic multidimensional array reference (`mdspan`) defines types and functions for mapping multidimensional indices in its *domain*, a multidimensional index space, to the `mdspan`'s *codomain*, elements of a contiguous span of objects. A *multidimensional index space* of *rank* is the Cartesian product $\prod_0 \times [0, N_1) \times \dots \times [0, N_{R-1})$ of half-open integer intervals. A *multidimensional index* is a element of a multidimensional index space. An `mdspan` has two policies: the *layout mapping* and the *accessor*. The layout mapping specifies the formula, and properties of the formula, for mapping a multidimensional index from the domain to an element in the codomain. The accessor is an extension point that allows modification of how elements are accessed. For example, [P0367] proposed a rich set of potential access properties.

A multidimensional array is not an array-of-array-of-array-of...

The multidimensional array abstraction has been fundamental to numerical computations for over five decades. However, the C/C++ language provides only a one-dimensional array abstraction which can be composed into array-of-array-of-array-of... types. While such types have some similarity to multidimensional arrays, they do not provide adequate multidimensional array functionality of this proposal. Two critical functionality differences are (1) multiple dynamic extents and (2) polymorphic mapping of multidimensional indices to element objects.

Optimized Implementation of Layout Mapping

We intend the layout mapping of a multidimensional index to be a constant-time `constexpr` operation that is trivially inlined and optimized when possible. Compiler vendors may apply optimizations such as loop invariant code motion, including partial evaluation of multidimensional index layout mappings when indices are loop invariant.

§ 3 Editing Notes

The proposed changes are relative to the working draft of the standard as of N4842.

The $\diamond$ character is used to denote a placeholder section number, table number, or paragraph number which the editor shall determine.

Add the header `<mdspan>` to the “C++ library headers” table in **[headers]** in a place that respects the table's current alphabetic order.

Add the header `<mdspan>` to the “Containers library summary” table in **[containers.general]** below the listing for `<span>`.

§ 4 Wording

The $\diamond$ character is used to denote a placeholder section number which the editor shall determine.

Make the following changes to 22.7.1 **[views.general]**,

- 1 The header `<span>` defines the view span. The header `<mdspan>` defines the view `basic_mdspan`, the type alias `mdspan`, and other facilities for interacting with these views.

Add the following subclauses to the end of the **[views]** subclause (after *span*):

22.7. $\diamond$ Header `<mdspan>` synopsis **[mdspan.syn]**

```
namespace std {
    // [mdspan.extents], class template extents
    template<size_t... Extents>
        class extents;

    // [mdspan.layout], Layout mapping policies
    class layout_left;
    class layout_right;
    class layout_stride;

    // [mdspan.accessor.basic]
    template<class ElementType>
```

```

class default_accessor;

// [mdspan.basic], class template mdspan
template<class ElementType, class Extents, class LayoutPolicy = layout_right,
        class AccessorPolicy = default_accessor<ElementType>>
class basic_mdspan;

template<class T, size_t... Extents>
using mdspan = basic_mdspan<T, extents<Extents...>>;

// [mdspan.submdspan]
template<class ElementType, class Extents, class LayoutPolicy,
        class AccessorPolicy, class... SliceSpecifiers>
constexpr basic_mdspan<see below> submdspan(const basic_mdspan<ElementType,
                                           Extents, LayoutPolicy, AccessorPolicy>&,
                                           SliceSpecifiers ...) noexcept;

// tag supporting submdspan
struct full_extent_t { explicit full_extent_t() = default; };
inline constexpr full_extent_t full_extent = full_extent_t{};
}

```

22.7.◆ Overview [mdspan.overview]

- 1 A *multidimensional index space* is a Cartesian product of integer intervals. Each interval can be represented by a half-open range $[I_b, I_e)$, where I_b and I_e are the lower and upper bounds of the i^{th} dimension. The *rank* of a multidimensional index space is the number of intervals it represents.
- 2 A *multidimensional index* is an element within the a multidimensional index space and can be represented as a pack of integer types. The multidimensional index `idx...` refers to an element within the domain of a multidimensional index space if both the following are true:
 - (2.1) — `sizeof...(idx)` is equal to rank, and
 - (2.2) — For all i in the range $[0, \text{rank})$, the i^{th} value of `idx` is in the range $[I_b, I_e)$.
- 3 For the following subsections, let r be a value in the range $[0, \text{rank})$.
- 4 `full_extent_t` is an empty class type.

22.7.◆ Class template extents [mdspan.extents]

22.7.◆.1 Overview [mdspan.extents.syn]

```

namespace std {

template<size_t... Extents>
class extents {
public:
    using size_type = size_t;

    // [mdspan.extents.cons], Constructors and assignment
    constexpr extents() noexcept = default;
    constexpr extents(const extents&) noexcept = default;
    constexpr extents& operator=(const extents&) noexcept = default;

    template<size_t... OtherExtents>
    constexpr extents(const extents<OtherExtents...>&) noexcept;
    template<class... SizeTypes>
    constexpr extents(SizeTypes...) noexcept;
    template<class SizeType>
    constexpr extents(const array<SizeType, rank_dynamic()>&) noexcept;
    template<size_t... OtherExtents>
    constexpr extents& operator=(const extents<OtherExtents...>&) noexcept;

    // [mdspan.extents.obs], Observers of the domain multidimensional index space
    static constexpr size_t rank() noexcept { return sizeof...(Extents); }
    static constexpr size_t rank_dynamic() noexcept
    { return ((Extents == dynamic_extent) + ...); }
    static constexpr size_type static_extent(size_t) noexcept
    { return rank() - rank_dynamic(); }
    constexpr size_type extent(size_t) const noexcept;

    // [mdspan.extents.compare], extents comparison operators
    template<size_t... OtherExtents>
    friend constexpr bool operator==(const extents&, const extents<OtherExtents...>&) noexcept;

private:
    static constexpr size_t dynamic_index(size_t) noexcept; // exposition only
    array<size_type, rank_dynamic()> dynamic_extents_{}; // exposition only
};

```

```
}
```

22.7.2 Overview [mdspan.extents.overview]

1 The class template `extents` represents a multidimensional index space of of rank equal to `sizeof...(Extents)`.

2 `extents<Extents...>` is a trivially copyable type.

3 E_r is a *dynamic extent* if it is equal to `dynamic_extent`, otherwise E_r is a *static extent*. For each E_r equivalent to `dynamic_extent`, the upper bound of the interval is stored in the exposition only array `dynamic_extents_` at `dynamic_extents_[dynamic_index(r)]`.

4 If E_r is a dynamic extent, let D_r be the value of `dynamic_extents_[dynamic_index(r)]`. The r^{th} interval of an `extents` is as follows:

(4.1) — $[0, E_r)$ if E_r is a static extent,

(4.2) — otherwise $[0, D_r)$.

```
constexpr size_t dynamic_index(size_t i) noexcept; // exposition only
```

5 — *Returns:* If `i <= sizeof...(Extents)` is true, returns the number of arguments before the i^{th} template parameter in the template parameter pack `Extents` equivalent to `dynamic_extent`. Otherwise, returns `rank_dynamic()`.

22.7.3 Constructors and assignment [mdspan.extents.cons]

```
template<size_t... OtherExtents>
constexpr extents(const extents<OtherExtents...>& other) noexcept;
```

1 — *Constraints:*

(1.1) — `sizeof...(OtherExtents)` equals `rank()`.

(1.2) — For all r where `static_extent(r) != dynamic_extent` and `other.static_extent(r) != dynamic_extent` are both true, `static_extent(r) == other.static_extent(r)` is true.

2 — *Expects:* For all r , `static_extent(r) == dynamic_extent || static_extent(r) == other.extent(r)` is true.

3 — *Effects:* For each r where `static_extent(r) == dynamic_extent` is true, assigns `other.extent(r)` to `dynamic_extent[dynamic_index(r)]`.

```
template<class... SizeTypes>
constexpr extents(SizeTypes... dynamic) noexcept;
```

4 — *Constraints:*

(4.1) — `(is_convertible_v<SizeTypes, size_type> && ...)` is true, and

(4.2) — `sizeof...(SizeTypes) == rank_dynamic()` is true.

5 — *Effects:* Initializes `dynamic_extents_` with `dynamic...`

```
template<class SizeType>
constexpr extents(const array<SizeType, rank_dynamic()> & dynamic) noexcept;
```

6 — *Constraints:* `is_convertible_v<SizeType, size_type>` is true.

7 — *Effects:* Initializes `dynamic_extents_` with `dynamic`.

```
template<size_t... OtherExtents>
constexpr extents& operator=(const extents<OtherExtents...>& other) noexcept;
```

8 — *Constraints:*

(10.1) — `sizeof...(OtherExtents)` equals `rank()`.

(10.2) — For all r where `static_extent(r) != dynamic_extent` and `other.static_extent(r) != dynamic_extent` are both true, `static_extent(r) == other.static_extent(r)` is true.

9 — *Expects:* For all r `static_extent(r) == dynamic_extent || static_extent(r) == other.extent(r)` is true.

10 — *Effects:* For each r where `static_extent(r) == dynamic_extent` is true, assigns `other.extent(r)` to `dynamic_extent[dynamic_index(r)]`.

11 — *Returns:* *this.

22.7.3 Observers of the domain multidimensional index space [mdspan.extents.obs]

```
constexpr size_type extent(size_t i) const noexcept;
```

- 1 — *Expects:* $i < \text{rank}()$ is true.
- 2 — *Returns:* `dynamic_extents_[dynamic_index(i)]` if `static_extent(i) == dynamic_extent` is true, otherwise `static_extent(i)`.

22.7.4 extents comparison operators [mdspan.extents.compare]

```
template<size_t... OtherExtents>  
friend constexpr bool operator==(const extents& lhs, const extents<OtherExtents...& rhs) noexcept;
```

- 1 — *Returns:* true if `lhs.rank()` equals `rhs.rank()` and `lhs.extents(r)` equals `rhs.extents(r)` for all r , otherwise false.

22.7. Layout mapping policy [mdspan.layout]

22.7.1 Layout mapping requirements [mdspan.layout.reqs]

1. A *layout mapping policy* is a class that contains a *layout mapping*, which is a nested class template.
2. A layout mapping policy and its layout mapping nested class template meet the requirements in Table [4](#).
3. A layout mapping meets the requirements of *Cpp17DefaultConstructible*, *Cpp17CopyAssignable*, and *Cpp17EqualityComparable*.
4. In Table [4](#):
 - MP denotes a layout mapping policy.
 - M denotes a specialization of the layout mapping policy's nested layout mapping template class.
 - E denotes a specialization of extents.
 - e denotes an object of type E .
 - m denotes an object of type M .
 - $i...$ and $j...$ are multidimensional indices in the multidimensional index space defined by e .
 - r is an integral value in the range $[0, e.\text{rank}())$.
 - $dr...$ is an integer pack where `sizeof...(dr) == e.rank()` is true and the r th element is equal to 1 and all other elements are 0.

Table [4](#) — Layout mapping policy and layout mapping requirements

Expression	Return Type	Returns	Expects
<code>MP::template mapping<E></code>	M		
<code>m.extents()</code>	E	<i>Returns:</i> e .	
<code>m(i...)</code>	$\text{E}::\text{size_type}$	<i>Returns:</i> A value in the range of $[0, \text{required_span_size}())$ defined by applying the layout mapping to a multidimensional index $i...$	<i>Expects:</i> $0 \leq \text{array}\{i...\}[r] < \text{extents().extent}(r)$ for all r in the range $[0, \text{Extents}::\text{rank}())$.
<code>m.required_span_size()</code>	$\text{E}::\text{size_type}$	<i>Returns:</i> If the multidimensional index space that e defines is empty, then zero, else 1 plus the maximum value of <code>m(i...)</code> for all $i...$ in e .	
<code>m.is_unique()</code>	bool	<i>Returns:</i> true if for every $i...$ and $j...$ where $i \neq j$... is true, <code>m(i...) != m(j...)</code> is true.	

<code>m.is_contiguous()</code>	<code>bool</code>	<i>Returns:</i> true if for all k in the range $[0, m.required_span_size())$ there exists an $i...$ such that $m(i...) equals k, otherwise false.$	
<code>m.is_strided()</code>	<code>bool</code>	<i>Returns:</i> true if for every r there exists an integer sr such that, for all $j...$ and $i...$ in e , where $j...$ equals $(i+dr)...$, $m(j...) - m(i...) equals sr. Otherwise, false.$	
<code>M::is_always_unique()</code>	<code>bool</code>	<i>Returns:</i> true if <code>m.is_unique()</code> is true for any object of type <code>M</code> .	
<code>M::is_always_contiguous()</code>	<code>bool</code>	<i>Returns:</i> true if <code>m.is_contiguous()</code> is true for any object of type <code>M</code> .	
<code>M::is_always_strided()</code>	<code>bool</code>	<i>Returns:</i> true if <code>m.is_strided()</code> is true for any object of type <code>M</code> .	
<code>m.stride(r)</code>	<code>E::size_type</code>	<i>Returns:</i> sr as defined in <code>m.is_strided()</code> above.	<i>Expects:</i> <code>m.is_strided()</code> is true.

22.7.2 Class template `layout_left` [`mdspan.layout.left`]

- 1 `layout_left` meets the requirements of layout mapping policy.
- 2 `layout_left` is a trivially copyable type. `layout_left::mapping<Extents>` is a trivially copyable type.
- 3 `layout_left` gives a layout mapping where the left-most extent is stride one and strides increase left-to-right as the product of extents.
- 4 If `Extents` is not a (possibly cv-qualified) specialization of `extents`, then the program is ill-formed.

```
namespace std {
struct layout_left {
    template<class Extents>
    class mapping {
    public:
        using size_type = typename Extents::size_type;

        constexpr mapping() noexcept = default;
        constexpr mapping(const mapping&) noexcept = default;
        constexpr mapping(const Extents&) noexcept;
        template<class OtherExtents>
            constexpr mapping(const mapping<OtherExtents>&) noexcept;

        constexpr mapping& operator=(const mapping&) noexcept = default;
        template<class OtherExtents>
            constexpr mapping& operator=(const mapping<OtherExtents>&) noexcept;

        constexpr Extents extents() const noexcept { return extents_; }

        constexpr size_type required_span_size() const noexcept;

    template<class... Indices>
        constexpr size_type operator()(Indices...) const noexcept;

    static constexpr bool is_always_unique() noexcept { return true; }
    static constexpr bool is_always_contiguous() noexcept { return true; }
    static constexpr bool is_always_strided() noexcept { return true; }

    constexpr bool is_unique() const noexcept { return true; }
    constexpr bool is_contiguous() const noexcept { return true; }
    constexpr bool is_strided() const noexcept { return true; }

    constexpr size_type stride(size_t) const noexcept;

    template<class OtherExtents>
        friend constexpr bool operator==(const mapping&, const mapping<OtherExtents>&) noexcept;

    private:
        Extents extents_{}; // exposition only
    };
};
}
```

22.7.2.1 `layout_left::mapping` members [`mdspan.layout.layout_left`]

```
constexpr mapping(const Extents& e) noexcept;
```

1 — *Effects*: Initializes extents_ with e.

```
template<class OtherExtents>
constexpr mapping(const mapping<OtherExtents>& other) noexcept;
```

2 — *Constraints*: is_convertible_v<OtherExtents, Extents> is true.

3 — *Effects*: Initializes extents_ with other.extents().

```
template<class OtherExtents>
constexpr mapping& operator=(const mapping<OtherExtents>& other) noexcept;
```

4 — *Effects*: Equivalent to:

```
extents_ = other.extents();
return *this;
```

```
constexpr size_type required_span_size() const noexcept;
```

5 — *Returns*: The product of extents().extent(r) for all r in the range [0, Extents::rank()).

```
template<class... Indices>
constexpr size_type operator()(Indices... i) const noexcept;
```

6 — *Constraints*:

(6.1) — sizeof...(Indices) == Extents::rank() is true, and

(6.1) — (is_convertible_v<Indices, size_type> && ...) is true.

7 — *Expects*: $0 \leq \text{array}\{i...\}[r] < \text{extents}().\text{extent}(r)$ for all r in the range [0, Extents::rank()).

8 — *Effects*: Let P... be the parameter pack such that is_same_v<make_index_sequence<size_type, sizeof...(Indices)>, integer_sequence<size_type, P...>> is true.
Equivalent to: return Extents::rank() > 0 ? (i*stride(P()) + ...) : 0;

```
constexpr size_type stride(size_t r) const;
```

9 — *Returns*: 1 if r equals zero, otherwise, the product of extents().extent(k) for all k in the range [0, r).

```
template<class OtherExtents>
friend constexpr bool operator==(const mapping& x, const mapping<OtherExtents>& y) noexcept;
```

10 — *Effects*: Equivalent to: return x.extents() == y.extents();.

22.7.3 Class template layout_right [mdspan.layout.right]

- 1 layout_right meets the requirements of layout mapping policy.
- 2 layout_right is a trivially copyable type. layout_right::mapping<Extents> is a trivially copyable type.
- 3 The layout mapping property layout_right gives a layout mapping where the right-most extent is stride one and strides increase right-to-left as the product of extents.
- 4 If Extents is not a (possibly cv-qualified) specialization of extents, then the program is ill-formed.

```
namespace std {
struct layout_right {
    template<class Extents>
    class mapping {
    public:
        using size_type = typename Extents::size_type;

        constexpr mapping() noexcept = default;
        constexpr mapping(const mapping&) noexcept = default;
        constexpr mapping(const Extents&) noexcept;
        template<class OtherExtents>
            constexpr mapping(const mapping<OtherExtents>&) noexcept;

        constexpr mapping& operator=(const mapping&) noexcept = default;
        template<class OtherExtents>
            constexpr mapping& operator=(const mapping<OtherExtents>&) noexcept;

        constexpr Extents extents() const noexcept { return extents_; }

        constexpr size_type required_span_size() const noexcept;

        template<class... Indices>
            constexpr size_type operator()(Indices...) const noexcept;
```

```

static constexpr bool is_always_unique() noexcept { return true; }
static constexpr bool is_always_contiguous() noexcept { return true; }
static constexpr bool is_always_strided() noexcept { return true; }

constexpr bool is_unique() const noexcept { return true; }
constexpr bool is_contiguous() const noexcept { return true; }
constexpr bool is_strided() const noexcept { return true; }

constexpr size_type stride(size_t) const noexcept;

template<class OtherExtents>
    friend constexpr bool operator==(const mapping&, const mapping<OtherExtents>&) noexcept;

private:
    Extents extents_{}; // exposition only
};
};
}

```

22.7.3.1 layout_right::mapping members [mdspan.layout.layout_right]

```
constexpr mapping(const Extents& e) noexcept;
```

- 1 — *Effects*: Initializes extents_ with e.

```
template<class OtherExtents>
    constexpr mapping(const mapping<OtherExtents>& other) noexcept;
```

- 2 — *Constraints*: is_convertible_v<OtherExtents, Extents> is true.

- 3 — *Effects*: Initializes extents_ with other.extents().

```
template<class OtherExtents>
    constexpr mapping& operator=(const mapping<OtherExtents>& other) noexcept;
```

- 4 — *Effects*: Equivalent to:

```
extents_ = other.extents();
return *this;
```

```
size_type required_span_size() const noexcept;
```

- 5 — *Returns*: The product of extents().extent(r) for all r in the range [0, Extents::rank()).

```
template<class... Indices>
    constexpr size_type operator()(Indices... i) const noexcept;
```

- 6 — *Constraints*:

- (6.1) — sizeof...(Indices) == Extents::rank() is true, and

- (6.1) — (is_convertible_v<Indices, size_type> && ...) is true.

- 7 — *Expects*: $0 \leq \text{array}\{\text{i} \dots\}[\text{r}] < \text{extents}().\text{extent}(\text{r})$ for all r in the range [0, Extents::rank()).

- 8 — *Effects*: Let P... be the parameter pack such that is_same_v<make_index_sequence<size_type, sizeof...(Indices)>, integer_sequence<size_type, P...>> is true.

Equivalent to: return Extents::rank() > 0 ? (i*stride(P()) + ...) : 0;

```
constexpr size_type stride(size_t r) const noexcept;
```

- 9 — *Returns*: 1 if r equals Extents::rank()-1, otherwise, the product of extents().extent(k) for all k in the range [r+1, Extents::rank()).

```
template<class OtherExtents>
    friend constexpr bool operator==(const mapping& x, const mapping<OtherExtents>& y) noexcept;
```

- 10 — *Effects*: Equivalent to: return x.extents() == y.extents();.

22.7.4 Class template layout_stride [mdspan.layout.stride]

- 1 layout_stride meets the requirements of layout mapping policy.

- 2 layout_stride is a trivially copyable type. layout_stride::mapping<Extents> is a trivially copyable type.

- 3 The layout mapping property layout_stride gives a layout mapping where the strides are user defined.

- 4 If Extents is not a (possibly cv-qualified) specialization of extents, then the program is ill-formed.

```
namespace std {
```

```

struct layout_stride {
    template<class Extents>
    class mapping {
    public:
        using size_type = typename Extents::size_type;

        constexpr mapping() noexcept = default;
        constexpr mapping(const mapping&) noexcept = default;
        constexpr mapping(const Extents&,
            const array<size_type, Extents::rank()>&) noexcept;
        template<class OtherExtents>
            constexpr mapping(const mapping<OtherExtents>&) noexcept;

        constexpr mapping& operator=(const mapping&) noexcept = default;
        template<class OtherExtents>
            constexpr mapping& operator=(const mapping<OtherExtents>&) noexcept;

        constexpr Extents extents() const noexcept { return extents_; }
        constexpr array<typename size_type, Extents::rank()> strides() const noexcept
        { return strides_; }

        constexpr size_type required_span_size() const noexcept;

    template<class... Indices>
        constexpr size_type operator()(Indices...) const noexcept ;

    static constexpr bool is_always_unique() noexcept { return true; }
    static constexpr bool is_always_contiguous() noexcept { return false; }
    static constexpr bool is_always_strided() noexcept { return true; }

    constexpr bool is_unique() const noexcept { return true; }
    constexpr bool is_contiguous() const noexcept;
    constexpr bool is_strided() const noexcept { return true; }

    constexpr size_type stride(size_t) const noexcept;

    template<class OtherExtents>
        friend constexpr bool operator==(const mapping&, const mapping<OtherExtents>&) noexcept;

    private:
        Extents extents_{}; // exposition only
        array<size_type, Extents::rank()> strides_{}; // exposition only
    };
};

```

22.7.4.1 layout_stride::mapping members [mdspan.layout.layout_stride]

```
constexpr mapping(const Extents& e, array<size_type, Extents::rank()> s) noexcept;
```

1 Let P be a permutation of the integers $0, \dots, \text{Extents::rank()} - 1$ and let p_i be the i^{th} element of P .

2 — *Expects:*

(2.1) — $s[i] > 0$ is true for all i in the range $[0, \text{Extents::rank}())$.

(2.2) — If $\text{Extents::rank}()$ is greater than zero, then there exists a permutation P such that $s[p_i] \geq s[p_{i-1}] * e.\text{extent}(p_{i-1})$ is true for all i in the range $[1, \text{Extents::rank}())$.

3 — *Effects:* Initializes `extents_` with e , and initializes `strides_` with s .

```
template<class OtherExtents>
constexpr mapping(const mapping<OtherExtents>& other) noexcept;
```

4 — *Constraints:* `is_convertible_v<OtherExtents, Extents>` is true.

5 — *Effects:* Initializes `extents_` with `other.extents()`, and initializes `strides_` with `other.strides()`.

```
template<class OtherExtents>
constexpr mapping& operator=(const mapping<OtherExtents>& other) noexcept;
```

6 — *Effects:* Equivalent to:

```
extents_ = other.extents();
return *this;
```

```
constexpr size_type required_span_size() const noexcept;
```

7 — *Returns:* The maximum of `extents().extent(r) * stride(r)` for all r in the range $[0, \text{Extents::rank}())$.

```
template<class... Indices>
constexpr size_type operator()(Indices... i) const noexcept;
```

8 — *Constraints:*

(8.1) — `sizeof...(Indices) == Extents::rank()` is true, and

(8.1) — `(is_convertible_v<Indices, size_type> && ...)` is true.

9 — *Effects:* `0 ≤ array{i...}[r] < extents().extent(r)` for all `r` in the range `[0, Extents::rank())`.

10 — *Effects:* Let `P...` be the parameter pack such that `is_same_v<make_index_sequence<size_type, sizeof...(Indices)>, integer_sequence<size_type, P...>>` is true.
Equivalent to: `return Extents::rank() > 0 ? (i*stride(P()) + ...) : 0;`

```
constexpr bool is_contiguous() const noexcept;
```

11 Let P be a permutation of the integers $0, \dots, \text{Extents}::\text{rank}() - 1$ and let p_i be the i^{th} element of P .

12 — *Returns:*

(12.1) — true if `Extents::ranks()` is zero.

(12.2) — Otherwise, true if there is a permutation P such that `min(stride( $p_i$ ))` equals one for i in the range `[0, Extents::rank())`, and `stride( $p_i$ )` equals `stride( $p_{i-1}$ ) * extents().extent( $p_{i-1}$ )` for i in the range `[1, Extents::rank())`.

(12.3) — Otherwise, false.

```
template<class OtherExtents>
friend constexpr bool operator==(const mapping& x, const mapping<OtherExtents>& y) noexcept;
```

13 — *Effects:* Equivalent to: `return x.extents() == y.extents();`.

22.7.◆ Accessor Policy [mdspan.accessor]

1 An *accessor policy* defines types and operations by which a contiguous set of objects are accessed.

22.7.◆.1 Accessor policy requirements [mdspan.accessor.reqs]

2 An accessor policy defines:

- (2.1) — a handle to a single element of type `element_type`;
- (2.2) — a handle to a contiguous set of elements of type `element_type`, accessible through the policy's access method;
- (2.3) — conversion of a handle to a contiguous set of elements, to a pointer `[conv.array]`; and
- (2.4) — getting a handle to the contiguous subset of elements beginning at an integer offset value.

3 *[Note: The type of reference need not be `element_type&`. The type of pointer need not be `element_type*`. — end note]*

4 An accessor policy meets the requirements of *Cpp17DefaultConstructible*, *Cpp17CopyAssignable*, and *Cpp17EqualityComparable*.

5 In Table ◆:

- (5.1) — A denotes an accessor policy.
- (5.2) — a denotes an object of type A .
- (5.3) — p denotes an object of type `A::pointer`.
- (5.4) — i and j each denote a `size_t` value.

Table ◆: Accessor policy requirements

Expression	Return Type	Requirements
A		A meets the requirements of <i>Cpp17DefaultConstructible</i> , <i>Cpp17CopyConstructible</i> , and <i>Cpp17CopyAssignable</i> .
<code>A::element_type</code>		<code>A::element_type</code> is required to be a complete object type that is not an abstract class type.
<code>A::pointer</code>		<code>A::pointer</code> meets the requirements of <i>Cpp17DefaultConstructible</i> , <i>Cpp17CopyConstructible</i> , and <i>Cpp17CopyAssignable</i> .

<code>A::reference</code>		<i>Constraints:</i> <code>is_convertible_v<A::reference,A::element_type></code> is true, and if <code>is_const_v<A::element_type></code> is false then <code>is_assignable_v<A::element_type,A::reference></code> is true.
<code>A::offset_policy</code>		Accessor policy for accessing a pointer returned by <code>a.offset(p,i)</code> . <i>Constraints:</i> — <code>A::offset_policy</code> meets the requirements of an accessor policy in Table ◆ , — <code>is_convertible_v<A, A::offset_policy></code> is true, and — <code>A::offset_policy</code> can be constructed from <code>a</code> .
<code>a.access(p, i)</code>	<code>A::reference</code>	<i>Returns:</i> An object which provides access to the <i>i</i> -th element in the range of elements that starts at <i>p</i> .
<code>a.offset(p, i)</code>	<code>A::offset_policy::pointer</code>	<i>Returns:</i> A pointer which references the <i>i</i> -th element in the range of elements that starts at <i>p</i> .

22.7.◆.2 Class template `default_accessor` [`mdspan.accessor.basic`]

- 1 `default_accessor` meets the requirements of accessor policy.
- 2 `ElementType` is required to be a complete object type that is neither an abstract class type nor an array type.

```
namespace std {
template<class ElementType>
struct default_accessor {
    using offset_policy = default_accessor;
    using element_type = ElementType;
    using reference = ElementType&;
    using pointer = ElementType*;

    constexpr typename offset_policy::pointer
        offset(pointer p, size_t i) const noexcept;

    constexpr reference access(pointer p, size_t i) const noexcept;
};
}
```

22.7.◆.2 Class template `default_accessor` members [`mdspan.accessor.members`]

```
constexpr typename offset_policy::pointer
offset(pointer p, size_t i) const noexcept;
```

- 1 — *Expects:* `p + i` is dereferenceable.
- 2 — *Returns:* `p + i`.

```
constexpr reference access(pointer p, size_t i) const noexcept;
```

- 3 — *Expects:* `p + i` is dereferenceable.
- 4 — *Returns:* `p[i]`.

22.7.◆ Class template `basic_mdspan` [`mdspan.basic`]

22.7.◆.1 `basic_mdspan` overview [`mdspan.basic.overview`]

- 1 `basic_mdspan` maps a multidimensional index in its domain to a reference to an element in its codomain span.
- 2 The *domain* of a `basic_mdspan` object is a multidimensional index space defined by an `extents`.
- 3 The *codomain* of a `basic_mdspan` object is a span of elements.
- 4 As with `span`, the storage of the objects in the codomain span of a `basic_mdspan` is owned by some other object.

```
namespace std {

template<class ElementType, class Extents, class LayoutPolicy, class AccessorPolicy>
class basic_mdspan {
public:

    // Domain and codomain types
    using extents_type = Extents;
    using layout_type = LayoutPolicy;
    using accessor_type = AccessorPolicy;
    using mapping_type = typename layout_type::template mapping_type<extents_type>;
```



```

using element_type = typename accessor_type::element_type;
using value_type = remove_cv_t<element_type>;
using size_type = size_t;
using difference_type = ptrdiff_t;
using pointer = typename accessor_type::pointer;
using reference = typename accessor_type::reference;

// [mdspan.basic.cons], basic_mdspan constructors, assignment, and destructor
constexpr basic_mdspan() noexcept = default;
constexpr basic_mdspan(const basic_mdspan&) noexcept = default;
constexpr basic_mdspan(basic_mdspan&&) noexcept = default;

template<class... SizeTypes>
    explicit constexpr basic_mdspan(pointer p, SizeTypes... dynamic_extents);
template<class SizeType, size_t N>
    explicit constexpr basic_mdspan(pointer p, const array<SizeType, N>& dynamic_extents);
constexpr basic_mdspan(pointer p, const mapping_type& m);
constexpr basic_mdspan(pointer p, const mapping_type& m, const accessor_type& a);
template<class OtherElementType, class OtherExtents, class OtherLayoutPolicy, class OtherAccessorPolicy>
    constexpr basic_mdspan(
        const basic_mdspan<OtherElementType, OtherExtents, OtherLayoutPolicy, OtherAccessorPolicy>& other);

constexpr basic_mdspan& operator=(const basic_mdspan&) noexcept = default;
constexpr basic_mdspan& operator=(basic_mdspan&&) noexcept = default;
template<class OtherElementType, class OtherExtents, class OtherLayoutPolicy, class OtherAccessorPolicy>
    constexpr basic_mdspan& operator=(
        const basic_mdspan<OtherElementType, OtherExtents, OtherLayoutPolicy, OtherAccessorPolicy>& other)
        noexcept;

// [mdspan.basic.mapping], basic_mdspan mapping domain multidimensional index to access codomain element
constexpr reference operator[](size_type) const noexcept;
template<class... SizeTypes>
    constexpr reference operator()(SizeTypes... indices) const noexcept;
template<class SizeType, size_t N>
    constexpr reference operator()(const array<SizeType, N>& indices) const noexcept;

accessor_type accessor() const { return acc_; }

static constexpr int rank() noexcept { return Extents::rank(); }
static constexpr int rank_dynamic() noexcept { return Extents::rank_dynamic(); }
static constexpr size_type static_extent(size_t r) noexcept { return Extents::static_extent(r); }

constexpr Extents extents() const noexcept { return map_.extents(); }
constexpr size_type extent(size_t r) const noexcept { return extents().extent(r); }
constexpr size_type size() const noexcept;
constexpr size_type unique_size() const noexcept;

// [mdspan.basic.codomain], basic_mdspan observers of the codomain
constexpr span<element_type> span() const noexcept;
constexpr pointer data() const noexcept { return ptr_; }

static constexpr bool is_always_unique() noexcept { return mapping_type::is_always_unique(); }
static constexpr bool is_always_contiguous() noexcept { return mapping_type::is_always_contiguous(); }
static constexpr bool is_always_strided() noexcept { return mapping_type::is_always_strided(); }

constexpr mapping_type mapping() const noexcept { return map_; }
constexpr bool is_unique() const noexcept { return map_.is_unique(); }
constexpr bool is_contiguous() const noexcept { return map_.is_contiguous(); }
constexpr bool is_strided() const noexcept { return map_.is_strided(); }
constexpr size_type stride(size_t r) const { return map_.stride(r); }

private:
    accessor_type acc_{}; // exposition only
    mapping_type map_{}; // exposition only
    pointer ptr_{}; // exposition only
};

```

- 5 `basic_mdspan<ElementType, Extents, LayoutPolicy, AccessorPolicy>` is a trivially copyable type if `AccessorPolicy`, `LayoutPolicy::mapping_type<Extents>` and `AccessorPolicy::pointer` are trivially copyable types.
`basic_mdspan<ElementType, Extents, LayoutPolicy, AccessorPolicy>` is a trivially default constructible type if `AccessorPolicy`, `LayoutPolicy::mapping_type<Extents>` and `AccessorPolicy::pointer` are trivially default constructible types.
- 6 `ElementType` is required to be a complete object type that is neither an abstract class type nor an array type.
- 7 If `Extents` is not a (cv-unqualified) specialization of `extents`, then the program is ill-formed.
- 8 If `LayoutPolicy` does not meet the layout mapping policy requirements, then the program is ill-formed.
- 8 If `AccessorPolicy` does not meet the accessor policy requirements or if `is_same_v<typename`

AccessorPolicy::element_type, ElementType> is false, then the program is ill-formed.

22.7.1 basic_mdspan constructors and assignment operators [mdspan.basic.cons]

```
template<class... SizeTypes>
explicit constexpr basic_mdspan(pointer ptr, SizeTypes... dynamic_extents);
```

1 — *Constraints:*

- (1.1) — (is_convertible_v<SizeTypes, size_type> && ...) is true,
- (1.2) — sizeof...(dynamic_extents) == rank_dynamic() is true,
- (1.3) — is_constructible_v<mapping_type, Extents> is true, and
- (1.4) — is_default_constructible_v<accessor_type> is true.

2 — *Effects:*

- (2.1) — Initializes ptr_ with ptr,
- (2.2) — Initializes map_ with Extents(dynamic_extents...), and

3 — *Throws:* Nothing.

```
template<class SizeType, size_t N>
explicit constexpr basic_mdspan(pointer p, const array<SizeType, N>& dynamic_extents);
```

4 — *Constraints:*

- (4.1) — is_convertible_v<SizeType, size_type> is true,
- (4.2) — N == rank_dynamic() is true,
- (4.3) — is_constructible_v<mapping_type, Extents> is true, and
- (4.4) — is_default_constructible_v<accessor_type> is true.

5 — *Effects:* Equivalent to: basic_mdspan(p, dynamic_extents[Rs]...), with Rs... from index_sequence<Rs...> matching make_index_sequence<N>.

6 — *Throws:* Nothing.

```
constexpr basic_mdspan(pointer p, const mapping_type& m);
```

7 — *Constraints:* is_default_constructible_v<accessor_type> is true.

8 — *Effects:*

- (8.1) — Initializes ptr_ with p,
- (8.2) — Initializes map_ with m, and

9 — *Throws:* Nothing.

```
constexpr basic_mdspan(pointer p, const mapping_type& m, const accessor_type& a);
```

10 — *Effects:*

- (10.1) — Initializes ptr_ with p,
- (10.2) — Initializes map_ with m, and
- (10.3) — Initializes acc_ with a.

11 — *Throws:* Nothing.

```
template<class OtherElementType, class OtherExtents, class OtherLayoutPolicy, class OtherAccessor>
constexpr basic_mdspan(const basic_mdspan<OtherElementType, OtherExtents, OtherLayoutPolicy,
OtherAccessor>& other);
```

12 — *Constraints:*

- (12.1) — is_convertible_v<OtherLayoutPolicy::template mapping<OtherExtents>, mapping_type> is true;
- (12.2) — is_convertible_v<OtherAccessor, Accessor> is true;
- (12.3) — is_convertible_v<OtherAccessor::pointer, pointer> is true;
- (12.4) — is_convertible_v<OtherExtents, extents_type> is true; and

- (12.5) — For all r in the range $[0, \text{rank}())$, if `other.static_extent(r) != dynamic_extent` && `static_extent(r) != dynamic_extent` is true, then `other.static_extent(r) == static_extent(r)` is true.
- 13 — *Expects:* For all r in the range $[0, \text{rank}())$, if `other.static_extent(r) == dynamic_extent` || `static_extent(r) == dynamic_extent` is true, then `other.extent(r) == extent(r)` is true.
- 14 — *Effects:*
- (14.1) — Initializes `ptr_` with `other.ptr_`,
- (14.2) — initializes `map_` with `other.map_`, and
- (14.3) — initializes `acc_` with `other.acc_`.
- 15 — *Throws:* Nothing.
- ```
template<class OtherElementType, class OtherExtents, class OtherLayoutPolicy, class OtherAccessor>
constexpr basic_mdspan& operator=(
 const basic_mdspan<OtherElementType, OtherExtents, OtherLayoutPolicy, OtherAccessor>& other);
```
- 16 — *Constraints:*
- (16.1) — `is_assignable_v<mapping_type, OtherLayoutPolicy::template mapping<OtherExtents>>` is true;
- (16.2) — `is_assignable_v<Accessor, OtherAccessor>` is true;
- (16.3) — `is_assignable_v<pointer, OtherAccessor::pointer>` is true;
- (16.4) — `OtherExtents::rank() == rank()` is true; and
- (16.5) — For all  $r$  in the range  $[0, \text{rank}())$ , if `other.static_extent(r) != dynamic_extent` && `static_extent(r) != dynamic_extent` is true, then `other.static_extent(r) == static_extent(r)` is true.
- 17 — *Expects:* For all  $r$  in the range  $[0, \text{rank}())$ , if `other.static_extent(r) == dynamic_extent` || `static_extent(r) == dynamic_extent` is true, then `other.extent(r) == extent(r)` is true.
- 18 — *Effects:*
- (18.1) — Assigns `other.ptr_` to `ptr_`,
- (18.2) — Assigns `other.map_` to `map_`, and
- (18.3) — Assigns `other.acc_` to `acc_`.
- 17 — *Throws:* Nothing.

## 22.7.2 basic\_mdspan members [mdspan.basic.members]

```
constexpr reference operator[](size_type i) const;
```

- 1 — *Constraints:* `rank() == 1` is true.
- 2 — *Expects:* `acc_.access(ptr_, map_(i))` shall be valid.
- 3 — *Effects:* Equivalent to: `return (*this)(i);`.

```
template<class... SizeTypes>
constexpr reference operator()(SizeTypes... indices) const;
```

- 4 — *Constraints:*
- (4.1) — `(is_convertible_v<SizeTypes, size_type> && ...)` is true, and
- (4.2) — `sizeof...(SizeTypes) == rank()` is true.
- 5 — *Expects:* `acc_.access(ptr_, map_(indices...))` shall be valid.
- 6 — *Effects:* Equivalent to: `return acc_.access(ptr_, map_(indices...));`.
- 7 — *Throws:* Nothing.

```
template<class SizeType, size_t N>
constexpr reference operator()(const array<SizeType, N>& indices) const;
```

- 8 — *Constraints:*
- (8.1) — `is_convertible_v<SizeType, size_type>` is true, and

- (8.2) — `rank() == N` is true.
- 9 — *Effects*: Equivalent to: `return apply(*this, indices);`.
- 10 — *Throws*: nothing.

```
constexpr size_type size() const noexcept;
```

- 11 — *Returns*: Product of `extent(r)` for all `r` in the range `[0, Extents::rank())`.

```
constexpr size_type unique_size() const noexcept;
```

- 12 — *Returns*: The number of unique elements in the codomain. *[Note: If `mapping().is_unique()` is true, this is identical to `size().`—end note]*

## 22.7. submdspan [mdspan.submdspan]

- 1 submdspan creates a basic\_mdspan with a domain that is a subset of the input basic\_mdspan's domain, and a codomain that is a subset of the input basic\_mdspan's codomain.
- 2 The SliceSpecifier template argument(s) and the corresponding value(s) of the arguments of submdspan after src determine the subset of src that the basic\_mdspan returned by submdspan views.

```
namespace std {
 // [mdspan.submdspan], submdspan creation
 template<class ElementType, class Extents, class LayoutPolicy,
 class AccessorPolicy, class... SliceSpecifiers>
 constexpr basic_mdspan<see below>
 submdspan(const basic_mdspan<ElementType, Extents, LayoutPolicy,
 AccessorPolicy>& src, SliceSpecifiers... slices) noexcept;
}
```

- 3 Let sub be the return value of `submdspan(src, slices...)`, let  $s_k$  be the  $k$ -th element of `slices...`, and let  $S_k$  be the type of the  $k$ -th element of `slices...`.
- 4 Define `map_rank` as an `array<size_t, src.rank()>` such that `map_rank[j]` equals `dynamic_extent` if `is_convertible_v<Sj, size_t>` is true, or else `map_rank[j]` equals the number of  $S_k$  with  $k < j$  such that `is_convertible_v<Sk, pair<size_t, size_t>> || is_convertible_v<Sk, full_extent_t>` is true.
- 5 Let `first` and `last` be exposition-only variables of type `array<size_t, src.rank()>`. For  $r$  in the range `[0, src.rank())`, define the values of `first[r]` and `last[r]` as follows:
- if `is_convertible_v<Sr, size_t>`, then `first[r]` equals  $s_r$ , and `last[r]` equals `first[r] + 1`;
  - otherwise, if `is_convertible_v<Sr, pair<size_t, size_t>>`, then `first[r]` equals `p.first`, and `last[r]` equals `p.second`, where `p` is the result of converting  $s_r$  to `pair<size_t, size_t>`;
  - otherwise, if `is_convertible_v<Sr, full_extent_t>`, then `first[r]` equals 0, and `last[r]` equals `src.extent(r)`.

### 6 Constraints:

- `sizeof(slices...)` equals `src.rank()`,
- `LayoutPolicy` is `layout_left`, `layout_right`, `layout_stride`, or any type in a possibly empty set of implementation-defined types, each of which meets the requirements of a layout mapping policy **[mdspan.layout.reqs]** *[Note: Implementation and user defined layout mapping policies could exist, for which taking an arbitrary submdspan does not make sense. — end note];* and
- For all  $k$  in the range `[0, src.rank())`, `is_convertible_v<Sk, size_t> || is_convertible_v<Sk, pair<size_t, size_t>> || is_convertible_v<Sk, full_extent_t>` is true.

### 7 Expects:

- For  $0 \leq r < \text{src.rank}()$ , `0 <= first[r] && first[r] <= last[r] && last[r] <= src.extent(r)` is true.

### 8 Ensures: All of the following:

- `sub.rank()` equals the number of  $k$  such that `is_convertible_v<Sk, pair<size_t, size_t>> || is_convertible_v<Sk, full_extent_t>` is true.
- Let the pack `i...` denote a multidimensional index in the domain of `src` with  $i_k$  denoting the  $k$ -th element of `i...`, such that  $i_k$  is greater than or equal to `first[k]` and  $i_k$  is less than `last[k]` for all  $k$  in the range `[0, src.rank())`. Let the pack `j...` denote a multidimensional index in the domain of `sub` with  $j_s$  denoting the  $s$ -th element of `j...`, such that  $j_s$  is equal to  $i_k$  minus `first[k]` where `map_rank[k]` equals  $s$  for all  $s$  in the range `[0, sub.rank())`. Then `sub(j...)` and `src(i...)` refer to the same element in the codomain of `src`.

- For  $0 \leq k < \text{src.rank}()$ , if  $\text{map\_rank}[k] \neq -1$  is true, then  $\text{sub.extent}(\text{map\_rank}[k])$  equals  $\text{last}[k] - \text{first}[k]$ .
- If  $\text{src.is\_strided}()$  is true, then  $\text{sub.is\_strided}()$  is true, and for all  $k$  in the range  $[0, \text{src.rank}())$ , if  $\text{map\_rank}[k] \neq -1$  is true, then  $\text{sub.stride}(\text{map\_rank}[k])$  equals  $\text{src.stride}(k)$ .
- For all  $k$  in the range  $[0, \text{src.rank}())$ , if  $\text{map\_rank}[k] \neq -1$  is true and  $\text{src.static\_extent}(k)$  does not equal  $\text{dynamic\_extent}$  and  $\text{is\_convertible\_v}<S_k, \text{full\_extent\_t}>$  is true, then  $\text{sub.static\_extent}(\text{map\_rank}[k])$  equals  $\text{src.static\_extent}(k)$ .

[Note: Example of submdspan use:

```
// Create a mapping
typedef extents<3,dynamic_extent,7> Extents3D;
layout_right::template mapping<Extents3D> map_right(10);

// Allocate a basic_mdspan
int* ptr = new int[3*8*10];
basic_mdspan<int,Extents3D,layout_right> a(ptr,map_right);

// Initialize the span
for(int i0=0; i0<a.extent(0); i0++)
 for(int i1=0; i1<a.extent(1); i1++)
 for(int i2=0; i2<a.extent(2); i2++)
 a(i0,i1,i2) = 10000*i0+100*i1+i2;

// Create Subspan
auto a_sub = submdspan(a,1,pair<int,int>(4,6),pair<int,int>(1,6));

// Print values of submdspan
for(int i0=0; i0<a_sub.extent(0); i0++) {
 for(int i1=0; i1<a_sub.extent(1); i1++) {
 cout << a_sub(i0,i1) << " ";
 }
 cout << endl;
}

/* Output
10401 10402 10403 10404 10405
10501 10502 10503 10504 10505
*/
```

- end note]

## § 5 Next Steps

We would like LEWG to poll on sending P0009 (‘mdspan’) to LWG for C++23 instead of C++ Library Fundamentals Technical Specification version 3, classified as an addition (P0592R4 bucket 3 item).

If P2128 Multidimensional subscript operator is also adopted into C++23, we will likely propose that `basic_mdspan::operator()` be replaced with `basic_mdspan::operator[]` for C++23.

## § 6 Implementation

There is an mdspan implementation available at <https://github.com/kokkos/mdspan/>.

## § 7 Related Work

LEWG issue

Previous paper:

- [[N4355]]

**P0860 : Access Policy Generating Proxy Reference**

The reference type may be a proxy for accessing an `element_type` object. For example, the *atomic* AccessorPolicy in **P0860** defines `AccessorPolicy::template accessor_type<T>::reference` to be `atomic_ref<T>` from **P0019**.

Related papers:

- **P0122** : span: bounds-safe views for sequences of objects The `mdspan` codomain concept of *span* is well-aligned with this paper.

- **P0367** : Accessors: The P0367 Accessors proposal includes polymorphic mechanisms for accessing the memory an object or span of objects. The `AccessorPolicy` extension point in this proposal is intended to include such memory access properties.
- **P0331** : Motivation and Examples for Multidimensional Array
- **P0332** : Relaxed Incomplete Multidimensional Array Type Declaration
- **P0454** : Wording for a Minimal `mdspan` Included proposed modification of `span` to better align `span` with `mdspan`.
- **P0546** : Preparing `span` for the future Proposed modification of `span`
- **P0856** : Restrict access property for `mdspan` and `span`
- **P0860** : atomic access policy for `mdspan`
- **P0900** : An Ontology of Properties for `mdspan`
- **P2128** : Multidimensional subscript operator
- **P2299** : `mdspan` and CTAD